

Function Pointers in C++

A Complete Guide from Every Angle

A function pointer is just like a variable pointer — but instead of pointing to **data**, it points to the **code (machine instructions)** of a function. Once you grasp this single idea, all of callbacks, dispatch tables, plug-in systems, and event handling fall into place.

1. The Core Concept

Compare a data pointer with a function pointer side by side:

```
int x = 5;
int* p = &x;                                // points to data in memory

void Print(int, char);
void (*fp)(int, char) = Print;               // points to FUNCTION CODE in memory
```

Memory layout:

```
Memory:
+-----+
| Code Segment |
| +-----+ |
| | Print function | @ 0x401234 <- address of the function
| |   for loop ... |      (where its machine code lives)
| |   cout << ch ... |
| +-----+ |
+-----+
| Stack |
| fp = 0x401234 | <- function pointer holds that address
+-----+
```

2. Why They Exist — The Real Need

A regular function call is **hardcoded at compile time**. The compiler bakes in `CALL 0x401234`. A function pointer call is **resolved at runtime** — whatever address the pointer holds right now gets called.

This unlocks:

- **Callbacks** — pass a function to be called later
- **Customization** — different behavior without changing code
- **Plug-in systems** — load and call functions discovered at runtime
- **Tables of choices** — dispatch tables, state machines
- **Event handlers** — 'when X happens, call this function'

3. The Syntax — How to Read It

```
void (*printPtr)(int, char) = Print;
//   ^         ^         ^
```

```
// | | +-- parameters the function takes
// | +-- pointer name (and the * makes it a pointer)
// +-- return type
```

Read it in this order:

```
void (*printPtr) (int, char)
A      A      A
|      |      |
+-- 3 -----+
|      |      |
+-- 1 -----+
|      |      |
+-- 2
```

1. printPtr is a POINTER (innermost)
2. ... to a function taking (int, char)
3. ... returning void

Practice reading these:

```
int (*fp)(int)           // fp: pointer to function(int) returning int
char (*fp)(int, int)     // fp: pointer to function(int,int) returning char
void (*fp)()             // fp: pointer to function() returning void
double (*fp)(double, double) // fp: pointer to function(double,double) returning double
```

Note: The parentheses around (*fp) are **mandatory**. Without them, the * binds to the return type and the declaration becomes a function returning a pointer instead of a pointer to a function.

```
void (*fp)(int); // POINTER to function taking int, returning void
void *fp(int);   // DIFFERENT - function taking int, returning void*
```

4. Your Code — Line by Line

```
void functionPointer()
{
    atexit(endMessage);           // line 19
    Print(10, '#');               // line 20
    void (*printPtr)(int, char) = Print; // line 21
    (*printPtr) (12, '+');         // line 22
    printPtr(10, 'S');             // line 23
    cout << "\nEnd message is: " << endl; // line 25
}
```

Line 19 — atexit(endMessage)

This is a real-world function-pointer use. You pass the function **itself** (its address) — **not** its result. Note the absence of parentheses after endMessage; adding them would call it now.

```
atexit(endMessage);
//      ^
// Passing the FUNCTION ITSELF (its address) - not calling it
//
// atexit's signature: int atexit(void (*func)())
// It stores the pointer; at program exit, registered functions run (LIFO order)

Print    -> the function (its address) - a pointer value
Print()  -> CALL the function - get a result
```

Line 20 — Print(10, '#')

A regular, direct, hardcoded call. The compiler emits a CALL to Print's address.

Line 21 — Create the pointer

```
void (*printPtr)(int, char) = Print;
//                        ^
// "Print" without () means the ADDRESS of Print.
// You could also write: = &Print (both are equivalent for functions)

void (*p1)(int,char) = Print; // implicit address-of
void (*p2)(int,char) = &Print; // explicit address-of
// Both store the same address - same machine code location.
```

Line 22 — Classic dereference call

```
(*printPtr)(12, '+');
// Treats printPtr like a regular pointer:
// *printPtr -> the function it points to
// then () -> call it
```

Line 23 — Modern direct call

```
printPtr(10, 'S');
// Function pointers can be called DIRECTLY - no need for *
// The compiler auto-dereferences.
```

All four of these are valid and do the same thing:

```
Print(10, '#');           // direct call
```

```
(*Print)(10, '#');      // dereference call (Print decays to pointer)
printPtr(10, '#');      // call through pointer (modern)
(*printPtr)(10, '#');   // call through pointer (classic)
```

5. Why Both Calling Forms Work

Functions in C/C++ are special: a function **name** auto-converts to its **address**, and a function **pointer** auto-converts back when called.

```
Print          = address
&Print         = address
*Print         = function (auto re-decays to pointer)
**Print        = same (decays again)
***Print       = same - you can keep going forever!
```

```
Print()        = call
(*Print)()     = call
(**Print)()    = call
(***Print)()   = call
```

This is a quirk of C - functions are NOT true first-class values.

6. Cleaner Syntax — typedef and using

The raw syntax is ugly. Use type aliases to make code readable:

```
// Hard to read
void (*printPtr)(int, char) = Print;
void (*signal(int, void (*)(int)))(int);    // real signal() signature

// typedef (old style)
typedef void (*PrintFunc)(int, char);
PrintFunc printPtr = Print;

// using (modern, C++11) - same meaning, easier to read
using PrintFunc = void (*)(int, char);
PrintFunc printPtr = Print;

// Use PrintFunc anywhere - like any type:
PrintFunc fp = Print;
void callTwice(PrintFunc fn) {             // pass as parameter - clean!
    fn(5, '*');
    fn(3, '!');
}
```

7. Function Pointers as Parameters — Callbacks

This is the #1 real-world use of function pointers.

```
// A function that takes another function as input
void doTwice(void (*action)(int)) {
    action(1);
    action(2);
}

void print(int x) { cout << "print: " << x << endl; }
void square(int x) { cout << "square: " << x*x << endl; }

int main() {
    doTwice(print);    // -> print: 1    print: 2
    doTwice(square);  // -> square: 1    square: 4
    // SAME doTwice, DIFFERENT behavior based on what you pass
}
```

STL algorithms use this pattern everywhere:

```
#include <algorithm>

bool descending(int a, int b) { return a > b; }
bool ascending (int a, int b) { return a < b; }

vector<int> v = {3, 1, 4, 1, 5, 9};
sort(v.begin(), v.end(), descending);    // pass function pointer
sort(v.begin(), v.end(), ascending);     // different sort by passing different fn
```

8. Arrays of Function Pointers — Dispatch Tables

```
// Calculator with operation table
double add (double a, double b) { return a + b; }
```

```

double sub (double a, double b) { return a - b; }
double mul (double a, double b) { return a * b; }
double divv(double a, double b) { return a / b; }

// Array of function pointers
double (*ops[4])(double, double) = { add, sub, mul, divv };
// Cleaner with using:
// using Op = double (*)(double, double);
// Op ops[4] = { add, sub, mul, divv };

int main() {
    cout << ops[0](10, 3);    // add -> 13
    cout << ops[1](10, 3);    // sub -> 7
    cout << ops[2](10, 3);    // mul -> 30
    cout << ops[3](10, 3);    // div -> 3.33

    int choice = 2;
    cout << ops[choice](5, 5); // -> 25 (mul)
    // No if/switch chain - pure data-driven dispatch.
}

```

9. Function Pointer as Return Type

```
// Function that RETURNS a function pointer
// Raw syntax - extremely ugly:
int (*getOperation(char op))(int, int) {
    if (op == '+') return add;
    if (op == '-') return sub;
    return nullptr;
}

// Same thing with using - readable:
using IntOp = int(*)(int, int);
IntOp getOperation(char op) {
    if (op == '+') return add;
    if (op == '-') return sub;
    return nullptr;
}

int main() {
    auto fn = getOperation('+');
    cout << fn(3, 4);    // 7
}
```

10. Function Pointers vs Modern C++ Alternatives

Feature	Function Pointer	std::function	Lambda
Syntax	void(*fp)(int)	function<void(int)>	[(int x){...}]
Captures vars?	No	Yes	Yes
Type-safe?	Limited	Yes	Yes
Performance	Fastest	Slower (heap maybe)	Often inlined
Class members?	Special syntax	Yes	Yes
Header needed	none	<functional>	none
C compatibility	Yes	No (C++ only)	No (C++ only)

Same task — three ways:

```
// 1. Function pointer (old style, fastest, C compatible)
void (*fp)(int, char) = Print;
fp(5, '*');

// 2. std::function (most flexible, can hold ANY callable)
#include <functional>
std::function<void(int, char)> sf = Print;
sf(5, '*');

// 3. Lambda (modern, can capture context)
auto lam = [](int n, char c) {
    for (int i = 0; i < n; i++) cout << c;
};
lam(5, '*');
```

11. Class Member Function Pointers — A Different Beast

```
class Car {
public:
    void honk() { cout << "Beep!"; }
    void brake() { cout << "Stop!"; }
};

// Special syntax - note the ClassName::
void (Car::*memFp)() = &Car::honk;
//      ^      ^      ^
//      class  member ptr  MUST use &Class::method (no auto-decay!)

Car c;
(c.*memFp)();    // call through object - note: .*
                // prints: Beep!

Car* pc = &c;
(pc->*memFp)();  // call through pointer - note: ->*
                // prints: Beep!
```

Note: Member function pointers are **not** the same as regular function pointers. They need their own special syntax with `.*` and `->*`.

12. Common Real-World Uses

```
// 1. atexit - your example
atexit(cleanup);    // call cleanup when program ends

// 2. qsort - C-style sort
int cmp(const void* a, const void* b) { return *(int*)a - *(int*)b; }
qsort(arr, n, sizeof(int), cmp);

// 3. signal handlers - OS signals
signal(SIGINT, [](int s){ cout << "Ctrl+C pressed!"; });

// 4. Event callbacks - GUI/game frameworks
button.onClick(handleClick);
window.onClose(saveData);

// 5. Thread function entry point
pthread_create(&t, nullptr, threadFunc, args);

// 6. Plugin systems - function loaded at runtime
auto fn = (void(*)())dlsym(handle, "plugin_init");

// 7. State machines
using StateFn = void(*)();
StateFn currentState = idleState;
currentState();    // dispatches to current state's handler
```

13. Complete Cheat Sheet

DECLARATION:

```
return_type (*name)(params) = function;
void (*fp)(int, char) = Print;
```

ASSIGNMENT:

```
fp = Print;           implicit address
fp = &Print;          explicit address - same result
```

CALLING (both work):

```
fp(args);             modern, clean
(*fp)(args);          classic, explicit dereference
```

CLEAN ALIAS:

```
using FnType = void(*) (int, char);
FnType fp = Print;
```

AS PARAMETER (callback):

```
void run(void (*cb)(int)) { cb(5); }
```

ARRAY OF FUNCTION POINTERS:

```
void (*table[4])(int) = { fnA, fnB, fnC, fnD };
table[i](42);
```

KEY RULES:

- Function name without () = address
- () around (*name) is MANDATORY
- Signatures must match EXACTLY (return + params + qualifiers)
- Can't bind to member functions (need Class::* syntax)

- Cannot capture variables (use `lambda/std::function` instead)

Output of YOUR Program

```
#####          <- Print(10, '#')
+++++++         <- (*printPtr)(12, '+')
SSSSSSSSSS      <- printPtr(10, 'S')

End message is:
Ending the program      <- atexit fired endMessage on program exit
```

***The big picture:** A function pointer turns a function from a hardcoded jump-target into a **value** you can store, pass, and choose at runtime. This single concept underlies callbacks, event systems, plug-ins, dispatch tables, and the entire `<algorithm>` library. Modern C++ adds `std::function` and `lambdas` as more powerful alternatives — but function pointers remain the lightest, fastest, and most C-compatible way to refer to 'a piece of code I want to call later.'*